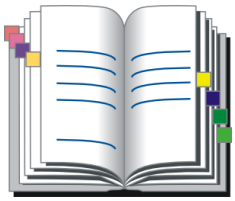
University of Nottingham
UK | CHINA | MALAYSIA

Faculty of Science and Engineering

Department of Electrical and Electronic Engineering

Project Activity Log Sheets



ACADEMIC YEAR 2025 / 2026

Course Code: H61AEE (EEEE1027) / H61AME (EEEE1031)

Student's Name : ……Tan Kai Le……..…….

Student's ID : ……20675369…….

Group Number : ……E34..…….

Course : Please tick (√)

Electrical and Electronic Engineering (√)
Mechatronic Engineering ( )

Student's Signature :

Project Activity Log

Date (and time): 27 January 2026, 5:02 PM

Objectives:

- Install Raspberry Pi 4
- Connect VNC IP address for remote control
- Connect L298N motor driver to Raspberry Pi 4

List of Components: keyboard, monitor , mouse, HDMI adapter, Raspberry Pi 4, L298N motor driver

Activities Done:

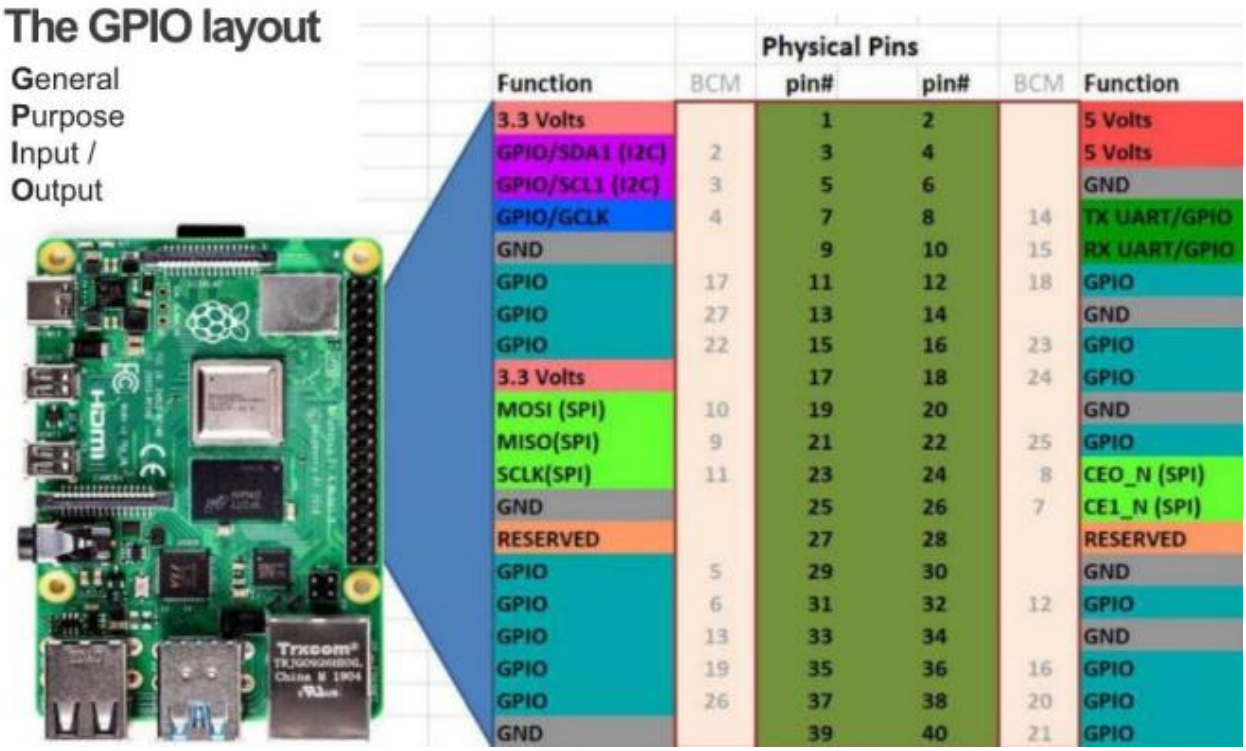
1.Raspberry Pi Initial Setup & VNC Configuration To begin the project, the Arduino from the Semester 1 car was removed to make way for the Fog Computing node.

Procedure:

1. The Raspberry Pi 4 was powered using its provided power supply and temporarily connected to a monitor using an HDMI adapter.
2. During this initial boot, a local user machine account was successfully created.
3. A mobile hotspot was activated to provide an internet connection to the Raspberry Pi.
4. The VNC server application was downloaded and enabled on the Raspberry Pi, and the corresponding VNC client was installed on the laptop.
5. By connecting both devices to the same mobile hotspot network, the local IP address of the Pi was retrieved and keyed into the laptop to establish a wireless remote control link.

2.Connect L298N motor driver to Raspberry Pi 4

With the headless setup complete, the L298N motor driver was physically wired to the Raspberry Pi. To match the specific GPIO.BOARD initialization parameters defined in the control software, the jumper wires were connected as follows:



Pinout:

- ENA_RIGHT: Pin 18
- IN1_RIGHT: Pin 15
- IN2_RIGHT: Pin 16
- ENB_LEFT: Pin 13
- IN3_LEFT: Pin 11
- IN4_LEFT: Pin 12

Ground connections were linked to ensure a common reference voltage.

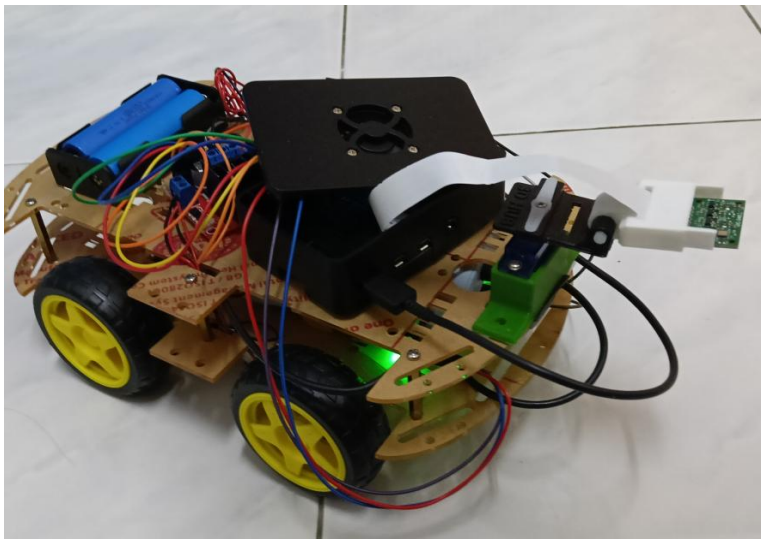


Image of fully assembled robot car.

Summary

- Raspberry Pi 4 installed
- VNC IP address connected for remote control
- Wired Raspberry Pi 4 to motor driver successfully

Date (and time): 29 January 2026, 5:02 PM

Project week 1

Objectives:

- control the robot car to go forward, backward, left and right
- turn 45, 60 ,90 degrees clockwise and anticlockwise
- measure distance using different PWM values
- find speed of robot car from a specific PWM value

Lists of Components: Robot car, Measuring tape, Protractor.

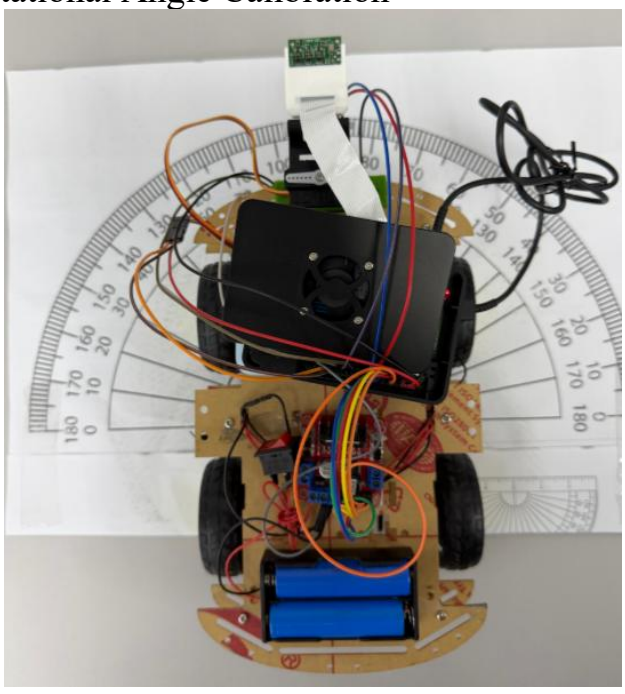
Activities Done:

Motor Control Logic & Testing

Procedure:

1. I developed a Python script utilizing the RPi.GPIO library to send signals to the L298N motor driver.
2. I created four primary functions move forward, move backward, turn left, and turn right to handle basic navigation.
3. For forward and backward movement, all four motors were programmed to spin in the same direction.
4. For turning, the code was configured to spin the left and right wheels in opposite directions, allowing the car to rotate on the spot.
5. I tested these commands by manually triggering them in the script to ensure the wiring was correct and the motors responded instantly to the code.

Rotational Angle Calibration



(image of robot car on protractor)

Procedure:

1. The robot was placed on a flat surface with a protractor aligned to its center axis.
2. The turning script was executed with a small time delay.
3. The time delay variable was iteratively adjusted until the physical turn exactly matched the target angle.
4. Distinct times were recorded for clockwise and anticlockwise rotations due to hardware variance.

Target Angle (Degrees)	Clockwise / Right Turn Time (s)	Anticlockwise / Left Turn Time (s)
45	0.315	0.400
60	0.400	0.510
90	0.520	0.600

Discussion:

The data reveals that the robot is physically asymmetrical. It takes longer to turn left than it does to turn right (for example, 0.60 seconds for a 90° left turn versus 0.52 seconds for a 90° right turn). This is likely caused by hardware flaws like uneven weight from the battery, friction in the motors, or slight voltage differences in the motor driver

2.Distance vs. PWM Calibration

Procedure:

1. The robot was placed at a fixed starting line on a flat surface.
2. The Python code was adjusted to run the motors for exactly 2 seconds at a specific PWM duty cycle.
3. The script was executed, and a measuring tape was used to record the total distance traveled.
4. This process was repeated for duty cycles ranging from 50% up to 100%.

PWM Duty Cycle (%)	Distance Traveled (cm)
50	33.0
60	47.0
70	55.0
75	56.5
80	56.5
90	64.5
100	68.0

Discussion:
The distance calibration table reveals a power plateau, where increasing the PWM from 75% to 80% resulted in no extra distance (staying at 56.5 cm). This shows that the motors have a limit where more power just causes heat or instability instead of more speed. By calculating velocity from the distance and time measurements, I found that the car reaches **26.11 cm/s** at an 80% duty cycle and tops out at **32 cm/s** at 100% duty cycle.

Summary:

- Tested the motor control and could manually command it to move accordingly.
- Successfully recorded the time to turn the car at exact angles in both directions
- Measured the distance for varying PWM values and calculated the speed of the robot car using 80% PWM.

Date (and time): 5 March 2026, 4:45 PM

Project week 2

Objectives

- Install the Picamera2
- Code a basic black line following algorithm using PID control.
- Develop a hybrid detection system utilizing AI for shapes and symbols, Template Matching for arrows.

Lists of Components: Raspberry Pi 4, Pi Camera2.

Activities Done:

1. **Vision Library Setup & Line Following Implementation**

Procedure:

1. With the Raspberry Pi connected to the internet, I used the terminal to install the Picamera2 library and its dependencies to allow Python to interface with the new camera hardware.
2. The library was imported into the main script, and the camera was initialized at a resolution of 640 by 480 pixels for real time streaming.
3. Live frames were captured and cropped to focus on the floor area directly in front of the robot.
4. Each frame was converted to Grayscale, blurred to remove pixel noise, and passed through an inverse binary threshold to isolate the black line from the floor.
5. OpenCV contour detection was used to find the largest black object, and its center point was calculated using Image Moments.
6. A PID control formula was applied to calculate the steering error based on the line's position, allowing the motors to make smooth corrections.

2. **Hybrid Detection System Integration**

Procedure:

1. A TFLite AI model was loaded into the script to classify complex symbols like the Recycle and Danger logos.
2. Grayscale reference images of arrows were loaded as templates for OpenCV.
3. A "Contour Gate" was programmed to skip heavy processing on empty frames; the AI and Template Matching only run if a candidate shape is physically detected in the frame first.

4. If a shape is detected, the AI model checks for symbols; if confidence is low, the image is passed to the Template Matcher to check for directional arrows.

Optimized Confidence Levels (Post-Calibration)

Symbol Detection Table

Symbol	**AI Confidence (%)**	**TMPM Confusion (Highest Match)**
Recycle	98.4	64.2% as **Right Arrow**
Danger	97.2	52.1% as **Up Arrow**
Pressbutton	95.8	58.7% as **Down Arrow**
Fingerprint	99.1	46.5% as **Right Arrow**
Qr code	99.4	41.3% as **Left Arrow**

Shape Detection Table

Shapes	**AI Confidence (%)**	**TMPM Confusion (Highest Match)**
Star	97.5	60.4% as **Left Arrow**
Diamond	98.2	74.1% as **Right Arrow**
Hospital/cross	94.3	76.8% as **Down Arrow**
Trapezoid	87.6	51.9% as **Left Arrow**
Pacman	96.9	44.2% as **Up Arrow**
Semicircle	98.1	54.8% as **Right Arrow**
Octagon	96.4	66.5% as **Down Arrow**

Arrow Detection Table

Arrows	**AI Confusion (Highest Match)**	**TMPM Confidence (%)**
Left	59.4% as **Octagon**	81.4
Right	62.8% as **Semicircle**	78.6
Up	34.1% as **Diamond**	85.1
Down	61.2% as **Trapezoid**	82.3

Black Line Following

Procedure:

1. Frame Pre-processing: The live frame was cropped to a 640x160 "Region of Interest" at the bottom of the feed to ignore background objects.

```
if not pause_this_frame:
    crop = frame_bgr[160:480, 0:640]
```

(code snippet for frame pre-processing)

2. Colour Isolation: I used cv2.cvtColor to convert to Grayscale and applied cv2.GaussianBlur to remove sensor noise.

```
gray = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5,5), 0)
_, mask_black = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
```

(code snippet for colour isolation)

3. Centroid Calculation: The code utilized Image Moments (cv2.moments) to find the horizontal center (cx) of the largest detected black contour.

```
M = cv2.moments(c)
if M['m00'] != 0:
    topmost = tuple(c[c[:, :, 1].argmin()][0])
    center_x = int(M['m10'] / M['m00'])

    cx = int((topmost[0] * 0.6) + (center_x * 0.4))
    error = 320 - cx
```

(code snippet for centroid calculation)

4. Error Correction: I programmed a PID loop where the steering correction was calculated as:

```
speed_penalty = abs(error) * 0.15
dynamic_speed = max(45, SPEED_FORWARD - speed_penalty)

integral += error
derivative = error - last_error
corr = (Kp * error) + (Ki * integral) + (Kd * derivative)
last error = error
```

(code snippet for error correction)

5. Motor Integration: This correction value was added to the left motor speed and subtracted from the right motor speed to create a smooth, differential turn.

```
l_spd = dynamic_speed + corr
r_spd = dynamic_speed - corr

if l_spd < 0:
    GPIO.output(IN3_LEFT, False); GPIO.output(IN4_LEFT, True)
else:
    GPIO.output(IN3_LEFT, True); GPIO.output(IN4_LEFT, False)

if r_spd < 0:
    GPIO.output(IN1_RIGHT, False); GPIO.output(IN2_RIGHT, True)
else:
    GPIO.output(IN1_RIGHT, True); GPIO.output(IN2_RIGHT, False)

pwm_left.ChangeDutyCycle(min(100, abs(l_spd)))
pwm_right.ChangeDutyCycle(min(100, abs(r_spd)))
```

(code snippet for motor integration)

PID table

Parameter	Value in Code	Physical Effect
Kp(Proportional)	**0.525**	**The Strength:** This makes the steering reactive. Because the track has sharp 90-degree turns, a high K_p of 0.525 ensures the car pulls hard back toward the line the moment it drifts.
Kd(Derivative)	**0.2**	**The Stabilizer:** This acts like a brake on the steering. Since the K_p is so high, the car would normally shake (oscillate). The K_d of 0.2 predicts when the car is reaching the centre and smooths out the movement.
Ki (Integral)	**0.00**	**The Offset:** This is used to correct long-term drifting (like a wheel being slightly stuck). In your testing, this was kept at 0.00 as the K_p and K_d were sufficient for the track layout.

Discussion: PID Performance

The near perfect navigation result was achieved by finding the correct ratio between the Kp and Kd values. With Kp at 0.525, the robot is aggressive enough to follow sharp curves without losing the line. However, aggressive steering usually leads to a zig-zag motion; I corrected this by tuning Kd to 0.2. This derivative term dampens the steering force as the error approaches zero, preventing the car from over-shooting the centre of the line. This specific combination allows the robot to maintain a stable heading even at the 45% base speed, providing a steady video feed for the detection systems to work accurately.

Summary

- Successfully installed the picamera2
- Used Template Matching and Teachable to detect shapes, symbols and arrows accurately
- Able to follow the centre black line closely throughout the whole track with PID control.

Date (and time): 9 April 2026, 2:30 PM

Project week 3

Objectives:

- navigate a prioritised multi-coloured track (Red, Yellow, and Black)
- 360° spin for Recycle symbol
- full stops for Danger and Press button symbols
- following direction of arrows (left, right and up except down)
- biometric symbols are to be printed on the screen

Activities Done

1. Integrated Multi-Colored Navigation: A priority system was developed to detect Red and Yellow lines first using HSV color masking. The Smart Priority Handoff logic was implemented, allowing the robot to automatically switch its focus from the black line to a colored junction if the detected colour area was large enough (>2000 pixels).

```
hsv = cv2.cvtColor(crop, cv2.COLOR_BGR2HSV)
mask_yellow = cv2.inRange(hsv, np.array([20, 100, 100]), np.array([35, 255, 255]))
mask_red = cv2.inRange(hsv, np.array([0, 120, 70]), np.array([10, 255, 255])) | cv2.inRange(hsv, np.array([170, 120, 70]), np.array([180, 255, 255]))
mask_color = mask_yellow | mask_red

gray = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5,5), 0)
_, mask_black = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

contours_c, _ = cv2.findContours(mask_color, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)

# SMART PRIORITY HANDOFF
if len(contours_c) > 0 and cv2.contourArea(max(contours_c, key=cv2.contourArea)) > 2000:
    mask_view = mask_color
else:
    mask_view = mask_black
```

(Code snippet for HSV and smart priority handoff)

2. Programmed Action Manoeuvres: Specific physical responses were coded for detected symbols. For arrows, a "Reverse-then-Pivot" sequence was implemented to ensure the robot centered itself on the junction before turning. For the Recycle logo, a high-torque 360-degree spin was programmed by setting both motors to maximum power and time.sleep() for the fixed duration of the turn.

```
if name in ["Left", "Right"]:
    move_backward(); time.sleep(0.3); stop_car()
    if name == "Left": turn_left()
    else: turn_right()
    time.sleep(0.2)
```

(code snippet for turning left and right functions)

```
elif "recycle" in best_symbol:
    print(f"\n*** AI PRIORITY TRIGGER: {best_symbol} ({best_conf*100:.1f}%) ***")
    GPIO.output(IN3_LEFT, True); GPIO.output(IN4_LEFT, False)
    GPIO.output(IN1_RIGHT, False); GPIO.output(IN2_RIGHT, True)
    pwm_left.ChangeDutyCycle(100); pwm_right.ChangeDutyCycle(100)
    time.sleep(1.8); stop_car()
    cooldown_until = time.time() + 2.5
    pause_this_frame = True; integral = 0
```

(Code snippet for recycle symbol)

3. Biometric Symbol Recognition: The AI logic was updated to recognize QR Code and Fingerprint symbols. When detected, these symbols were scanned and the results were printed on the Python terminal, displaying the symbol name and the confidence score to verify the robot's decisions during autonomous runs.

```
# --- FINGERPRINT & QR CODE TRIGGER ---
elif any(x in best_symbol for x in ["fingerprint", "qr"]):
    print(f"\n*** AI PRIORITY TRIGGER: {best_symbol} ({best_conf*100:.1f}%) ***")
```

(Code snippet for biometric symbol detection)

4. Software Performance Optimization: To keep the steering responsive, the main loop was optimized to run the AI "brain" only every 3rd frame (frame_counter%3 == 0). This prevented the Raspberry Pi from lagging and ensured the steering stayed smooth at a 45% base speed.

```
if not matched_template and not pause_this_frame and (frame_counter % 3 == 0):
    try: # <--- THIS IS THE TRY BLOCK THAT WAS MISSING ITS EXCEPT
        frame_rgb = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB)
        resized_ai = cv2.resize(frame_rgb, (224, 224))
        input_data = np.expand_dims(resized_ai, axis=0)

        if input_details[0]['dtype'] == np.float32:
            input_data = (np.float32(input_data) / 127.5) - 1.0
        else:
            input_data = np.uint8(input_data)
```

(Code snippet for software performance optimization)

5. Timing & Cooldown Calibration: A 2.5-second cooldown timer was established after any symbol action. This stopped the robot from getting "stuck" on a sign and repeating the same action before it had physically driven past the sticker.

```
cooldown_until = time.time() + 2.5
pause_this_frame = True; integral = 0
```

(Code snippet for timing and cooldown calibration)

Summary

- Successfully combined the detection system with the line-following algorithm using multithreading to ensure smooth autonomous navigation.
- Developed and calibrated custom movements for all required symbols and arrows, including a timed 360-degree rotation.
- Implemented a robust color-masking priority system to navigate crossroads accurately.

Everything was guided by Gemini AI